

BUILD A BANKING TRANSACTION LAKEHOUSE

By Lindokuhle Nyoka | June 2026

```

databricks
Free Edition

workspace

Banking Lakehouse Pipeline x +

Python ☆

3 minutes ago (2s) 27

("total_amount").collect()[0][0]
amounts_match = abs(float(silver_total) - float(gold_total)) < 0.01
checks.append(("Gold totals match Silver", amounts_match, f"Silver=
{silver_total}, Gold={gold_total}"))

print("=== PIPELINE HEALTH REPORT ===")
print("-" * 50)
for check_name, passed, detail in checks:
    status = "PASS" if passed else "FAIL"
    print(f"[{status}] {check_name} ({detail})")
print("-" * 50)
all_passed = all(passed for _, passed, _ in checks)
print(f"Overall: {'ALL CHECKS PASSED' if all_passed else 'SOME CHECKS
FAILED'}")

> See performance \(4\) ✓ Checked for improvements

=== PIPELINE HEALTH REPORT ===
-----
[PASS] Bronze has data (30 rows)
[PASS] Silver has no null IDs (0 nulls)
[PASS] Gold totals match Silver (Silver=13305.49, Gold=13305.49)
-----
Overall: ALL CHECKS PASSED

Delta Lake Time Travel
```

Building a Banking Transaction Lakehouse

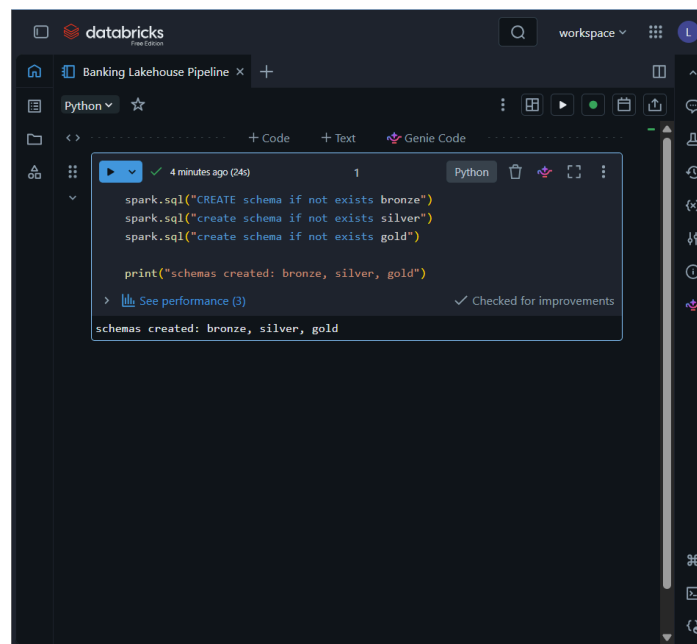
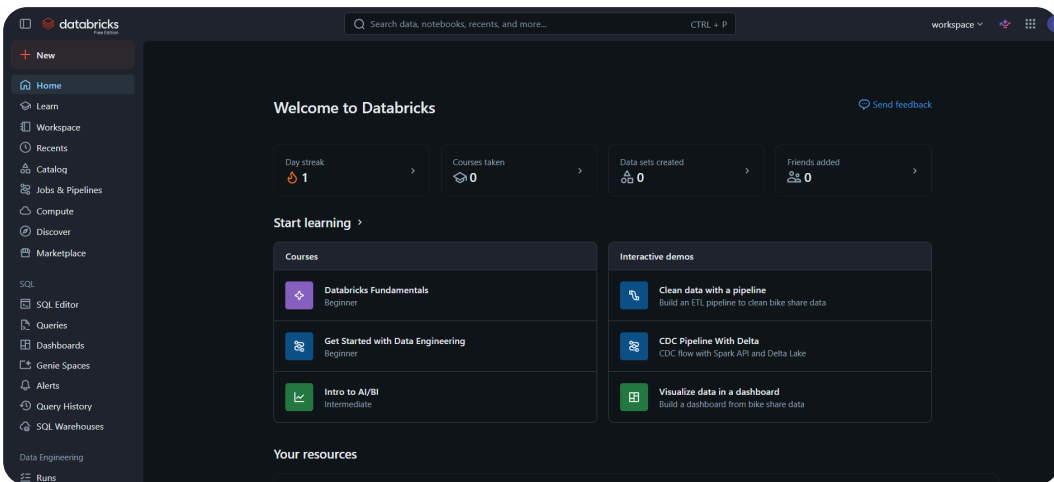
Project overview and architecture

In this project, I'm building a full medallion architecture pipeline. This matters for banking because they need a system to transform raw data into clean, trustworthy information.

Setting up Databricks Free Edition

In this step, I'm setting up all my software (Databricks) so I can run my pipeline on it.

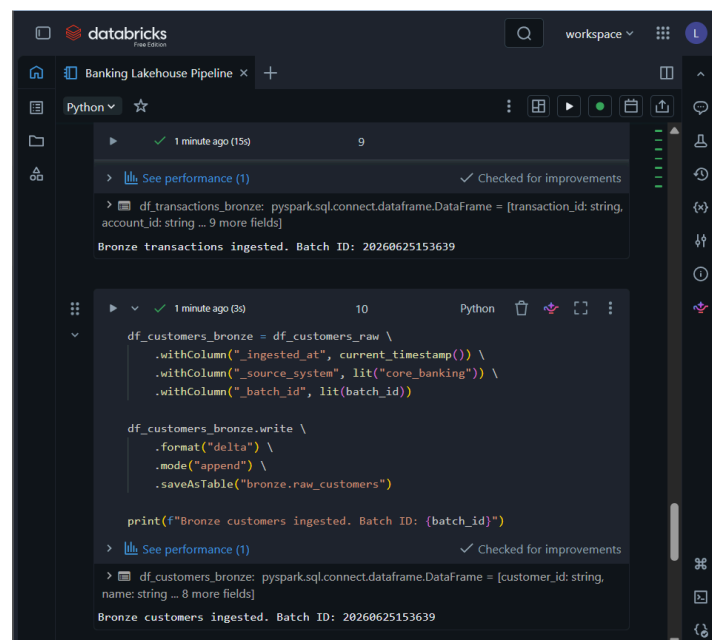
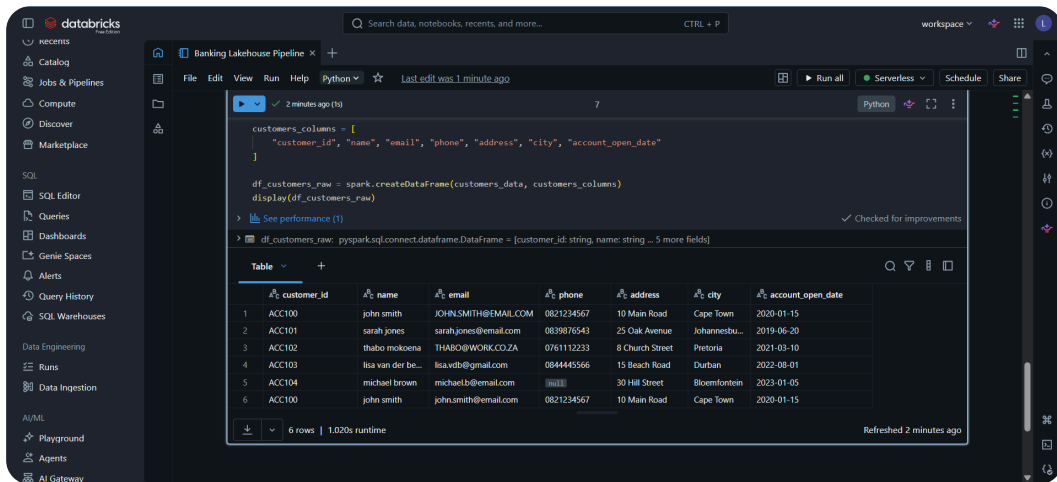
Configuring the Databricks Environment



Ingesting Raw Data into the Bronze Layer

Append-only ingestion with metadata

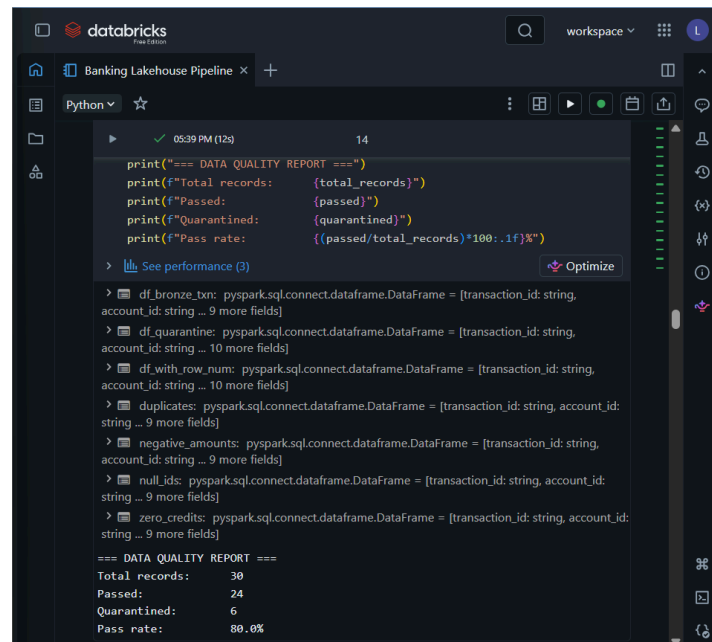
In this step, I'm building a Bronze Layer, so that I can store all the data.



Cleaning and Validating Data in the Silver Layer

Data quality checks and MERGE INTO upserts

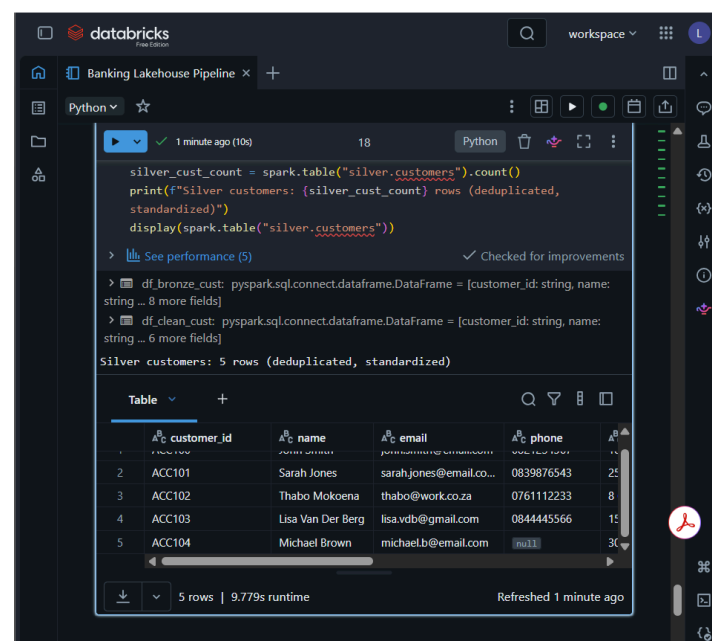
In this step, I'm cleaning data from the Bronze layer, so that the data is ready for transformation.



The screenshot shows a Databricks workspace with a Python script for data quality checks. The script prints a report with the following output:

```
print("=== DATA QUALITY REPORT ===")
print(f"Total records:      {total_records}")
print(f"Passed:                  {passed}")
print(f"Quarantined:             {quarantined}")
print(f"Pass rate:                {(passed/total_records)*100:.1f}%")
```

The output shows 30 total records, 24 passed, 6 quarantined, and a pass rate of 80.0%.



The screenshot shows a Databricks workspace with a Python script for data cleaning. The script prints a report with the following output:

```
silver_cust_count = spark.table("silver_customers").count()
print(f"Silver customers: {silver_cust_count} rows (deduplicated, standardized)")
display(spark.table("silver_customers"))
```

The output shows 5 rows (deduplicated, standardized).

customer_id	name	email	phone
ACC101	Sarah Jones	sarah.jones@email.co...	0839876543
ACC102	Thabo Mokoena	thabo@work.co.za	0761112233
ACC103	Lisa Van Der Berg	lisa.vdb@gmail.com	0844445566
ACC104	Michael Brown	michael.b@email.com	null

Building Business-Ready Aggregations in the Gold Layer

Star schema dimensional model

In this step, I'm building the gold layer so that business teams can query and analyse data more easily.

The screenshot shows a Databricks notebook titled "Banking Lakehouse Pipeline". The code cell is running Python and has just completed execution. The code defines a summary table with transaction dates and aggregates transaction counts and amounts. The output shows a table with 4 rows of summary data.

```
.format("delta") \
.mode("overwrite") \
.saveAsTable("gold.fact_daily_transactions")

print("Gold: Daily transaction summary created")
display(spark.table("gold.fact_daily_transactions"))
```

Gold: Daily transaction summary created

	transaction_date	total_transactions	total_amount	avg.
1	2024-03-01	4	5039.99	
2	2024-03-02	4	5915.50	
3	2024-03-03	1	200.00	
4	2024-03-04	2	2150.00	

4 rows | 8.113s runtime

The screenshot shows a Databricks notebook titled "Banking Lakehouse Pipeline". The code cell is running Python and has just completed execution. The code defines a fact table with customer SKs, account IDs, transaction counts, and total spend. The output shows a table with 4 rows of fact data.

```
.saveAsTable("gold.fact_customer_transactions")

print("Gold: Customer transaction facts created (star schema)")
display(spark.table("gold.fact_customer_transactions"))
```

Gold: Customer transaction facts created (star schema)

	customer_sk	account_id	transaction_count	total_spend
1	4	ACC103	1	
2	1	ACC100	4	
3	2	ACC101	3	1
4	3	ACC102	3	10

4 rows | 6.706s runtime

Enforcing Data Quality Gates and Audit Trails

Pipeline health assertions across all layers

In this step, I'm adding quality gates and time travel, so that I can query how the data looked like at any point in history.

Delta Lake time travel and audit history

DESCRIBE HISTORY showed me versions, timestamps, user details, and MERGE and CREATE TABLE. This is useful because it is useful for recreating historical reports, and auditing fraud investigations

Reflections and Takeaways

Key tools and concepts

The key tools I used include PySpark, python and Delta Lake, star schema. Key concepts I learnt include:

Building a complete medallion architecture pipeline (Bronze/Silver/Gold) that ingests raw banking transactions, cleans and deduplicates them with PySpark transformations, and produces business-ready aggregations.

Implementing Delta Lake MERGE INTO for incremental upsert processing with data quality gates, quarantine logic for bad records, and time travel for full audit history.

Designing a star schema dimensional model with fact and dimension tables in the Gold layer, ready for downstream analytics and reporting.

Time and challenges

This project took me approximately 2 hours. The most challenging part was understanding the code as I have little experience with PySpark and python.

I did this project today to learn how to be a data engineer and build a full medallion architecture pipeline. Another skill I want to learn is to be proficient in Python and PySpark. I also want to build pipelines without using AI for assistance.